Information Retrieval Ranking Models

Lecture 3: Sparse Ranking Methods (BM25, Language Models, Query Expansion)

Avishek Anand

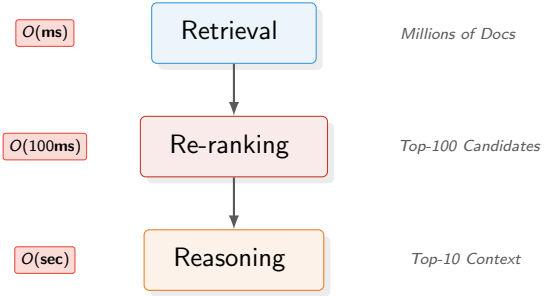
Information Retrieval

March 2, 2026

Outline

1. From Vector Space to Probabilistic Ranking
2. BM25: The Probabilistic Standard
3. Statistical Language Models for Ranking
4. Query Expansion and Pseudo Relevance Feedback
5. LLM-Based Query Expansion
6. Connections to Future Lectures

Where We Are: The IR Pipeline



Our Focus Today

The methods that **power the retrieval step** in this pipeline — the scoring functions that decide which documents make it to re-ranking at all.

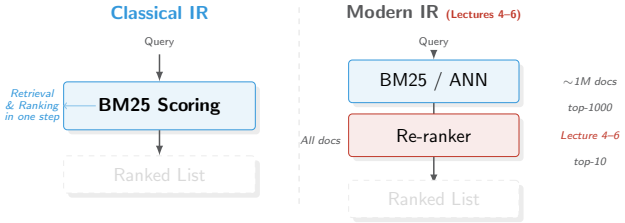
Classical vs. Modern: The Retrieval–Ranking Boundary

Classical IR

- **Retrieval = Ranking**
- One algorithm (BM25) does it all
- Query → BM25 score → ranked list

Modern IR (Lectures 4–6)

- **Retrieval ≠ Ranking**
- Stage 1: BM25/ANN → top-1000 (fast)
- Stage 2: Re-ranker/LTR → top-10 (accurate)
- Each stage only sees what the previous provides



Section 1

From Vector Space to Probabilistic Ranking

Recap: What We Know

Vector Space Model (Lecture 1)

- Documents and queries as vectors in term space
- Similarity measured by cosine distance
- Foundation for understanding term-based retrieval

Indexing (Lecture 2)

- Inverted index: term $\rightarrow$ posting list (docID, tf, positions)
- Accumulator-based scoring: only touch documents with term overlap
- Dense indexing: IVF+PQ for semantic vectors

Today's Question

We can store and retrieve efficiently. But **how do we score?**

The Probability Ranking Principle (PRP)

Theorem (Robertson, 1977)

If documents are ranked by decreasing probability of relevance given the query, then the expected effectiveness of the system is maximized—under the assumption of independent relevance.

Formally: Rank by $P(\text{relevant} \mid q, d)$

Implications:

- Gives a **theoretical justification** for probabilistic scoring
- All scoring functions (TF-IDF, BM25, LM) are approximations of this ideal
- Independence assumption is strong—violated in practice (documents are correlated)

Key Insight

The PRP tells us *what* to optimize. BM25 and Language Models tell us *how*.

The Ranking Problem

Formal Definition

Given:

- Query q (sequence of terms)
- Document collection D
- Inverted index (from Lecture 2)

Return: Ranked list scored by relevance $P(\text{relevant}|q, d)$

Term Weighting: TF-IDF

Term Frequency (TF)

- More occurrences $\rightarrow$ more important
- $\text{tf}(t, d) = \text{count}(t, d)$

Inverse Document Frequency (IDF)

- Rare terms $\rightarrow$ more discriminative
- $\text{idf}(t) = \log \frac{N}{\text{df}_t}$

Combined Score

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

TF-IDF Example

Query: "information retrieval" ($N = 100{,}000$ documents)

Term	Doc 1	Doc 2	df	IDF	Weight
information	3	1	5000	1.30	High TF
retrieval	2	0	500	2.30	High IDF
the	20	150	50000	0.30	Low IDF

Problem

Linear TF growth unrealistic:

- 100 occurrences $\neq$ 100$\times$ more relevant than 1
- Need saturation

Section 2

BM25: The Probabilistic Standard

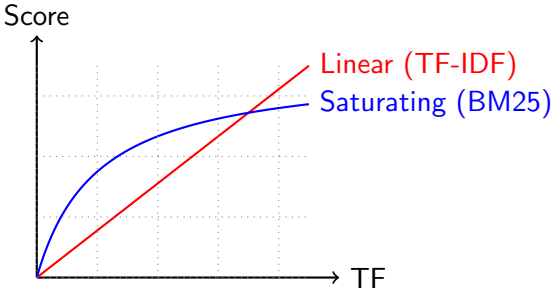
Motivation: TF Saturation

Observation:

- Relevance increases with TF
- But saturates quickly

Example:

- "algorithm" 5× vs. 50×
- 5→10 is meaningful
- 45→50 barely noticeable



Need

Non-linear TF component that saturates gracefully

BM25 Formula

Complete Formula

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

Three components:

1. **IDF:** Term importance (like TF-IDF)
2. **Saturating TF:** Non-linear term frequency
3. **Length normalization:** Penalize long documents

Standard parameters: $k_1 = 1.2$, $b = 0.75$

BM25 Component 1: IDF

IDF Formula

$$\text{IDF}(t) = \log \frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}$$

where:

- N = total number of documents
- df_t = document frequency of term t

Properties:

- Similar to TF-IDF but probabilistically motivated
- Can be negative for very common terms ($\text{df} > N/2$)
- Rare terms get high positive IDF
- Common terms get low or negative IDF

BM25 Component 2: Saturating TF

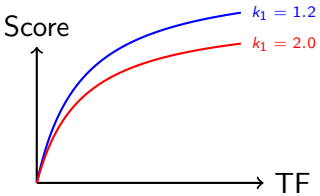
Saturating TF Formula

$$\frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1}$$

Parameter k_1 (typically 1.2–2.0):

- $k_1 \to 0$: Boolean matching (TF doesn't matter)
- $k_1 \to \infty$: Linear TF (like TF-IDF)
- $k_1 = 1.2$: Moderate saturation (standard)

TF	$k_1 = 1.2$	$k_1 = 2.0$
1	0.55	0.67
5	1.53	1.67
10	1.79	1.83
50	1.98	1.96



BM25 Component 3: Length Normalization

Length Normalization Term

$$1 - b + b \cdot \frac{|d|}{\text{avgdl}}$$

Parameter b (typically 0.75):

- $b = 0$: No length normalization
- $b = 1$: Full normalization (penalize long docs heavily)
- $b = 0.75$: Partial normalization (standard)

Why needed:

- Long documents have more terms just by chance
- Should we penalize or not? Depends on collection
- b controls the trade-off

BM25 Example: Effect of Document Length

Scenario: Term appears 20 times, $k_1 = 1.2$, $b = 0.75$, avgdl=1000

Short Document (500 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{500}{1000} \\ &= 0.625 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 0.625} \\ &= \frac{44}{20.75} = 2.12 \end{aligned}$$

Long Document (2000 words)

$$\begin{aligned} \text{norm} &= 1 - 0.75 + 0.75 \cdot \frac{2000}{1000} \\ &= 1.75 \\ \text{TF component} &= \frac{20 \cdot 2.2}{20 + 1.2 \cdot 1.75} \\ &= \frac{44}{22.1} = 1.99 \end{aligned}$$

Observation

Same TF, but short document scores higher due to length penalty

Worked Example

For a step-by-step end-to-end BM25 calculation, see the **Appendix** at the end of these slides.

BM25 Example: Rare vs. Common Terms

Collection: $N = 100{,}000$ documents

Term	df	IDF	Interpretation
quantum	100	6.21	Very discriminative
algorithm	5,000	2.99	Moderately discriminative
system	50,000	-0.41	Not discriminative
the	99,000	-6.21	Actively harmful

Key Insight

IDF difference drives discrimination: Rare terms dominate scoring (Matches on "quantum" worth $\sim 15\times$ matches on "system"). Common terms contribute little or negatively

Section 3

Statistical Language Models for Ranking

A Different Ranking Philosophy

BM25 Approach:

- "How well does document match query?"
- Scoring based on term overlap
- Weighted by importance (IDF, TF)

Matching Score

$$\text{score}(q, d) = \sum_t w(t, q, d)$$

Language Model Approach:

- "How likely would this document generate this query?"
- Rank by $P(q|d)$
- Probabilistic generative model

Generative Probability

$$\text{score}(q, d) = P(q|d)$$

A Different Ranking Philosophy

BM25 Approach:

- "How well does document match query?"
- Scoring based on term overlap
- Weighted by importance (IDF, TF)

Language Model Approach:

- "How likely would this document generate this query?"
- Rank by $P(q|d)$
- Probabilistic generative model

Key Shift

BM25 asks *"does this document contain the right words?"*
Language Models ask *"would a reader of this document ever write this query?"*

The Generative Story

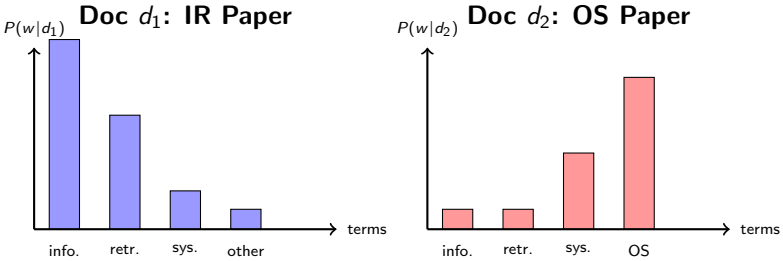
Intuition

Imagine randomly sampling words from a document. The document that is **most likely to produce your query terms** is ranked highest.

How it works step by step:

1. Pick a document d from the collection
2. The document defines a **language model** — a probability distribution $P(w|d)$ over words
3. **Sample** query words from this distribution: $P(q|d) = \prod_{w \in q} P(w|d)$
4. Rank documents by $P(q|d)$ — those more likely to "emit" the query rank higher

Visualizing Language Models



Concept: Each document is a probability distribution over the entire vocabulary. Retrieval asks: *"How likely is the query sequence under this distribution?"*

Query Likelihood Model: Philosophy

The Generative Story

Instead of matching keywords, we ask: **"What is the probability that this document would produce this query?"**

Mathematical Formulation:

- Assume query terms $w \in q$ are sampled **independently** from the document d (unigram assumption).
- Rank documents by:

$$P(q|d) = \prod_{w \in q} P(w|d)$$

- In practice, we use log-probabilities to avoid underflow:

$$\text{score}(q, d) = \sum_{w \in q} \log P(w|d)$$

Estimation

Maximum Likelihood Estimate (MLE)

How do we calculate $P(w|d)$? The simplest way is term frequency:

$$P_{\text{ML}}(w|d) = \frac{\text{count}(w, d)}{|d|}$$

But what is the catch here?

The Catch

If a query term w is **not in the document**, then $P_{\text{ML}}(w|d) = 0$.

- The product $P(q|d) = 0$ for the entire query!
- One missing term kills the score, even for a 90% relevant document.

Example: Query "machine learning algorithms" misses a document about "machine learning" because the word "algorithms" is absent.

Zero Probability Example

Query: "machine learning algorithms"
Document: "This paper presents a novel neural network approach to supervised machine learning." (15 words)

Term	**count(w,d)**	$P_{\mathbf{ML}}(w\|d)$	**Result**
machine	1	$1/15 = 0.067$	✓
learning	1	$1/15 = 0.067$	✓
algorithms	0	$0/15 = 0$	✗ ZERO!
$P(q\|d)$	$0.067 \times 0.067 \times 0 = \mathbf{0}$		

Problem

Document is clearly relevant, but one missing term kills the entire score!

Smoothing: Core Idea

Principle

Mix document evidence with collection evidence:

$$P(w|d) = f(\text{document evidence}, \text{collection evidence})$$

Collection Language Model:

$$P(w|C) = \frac{\sum_{d' \in C} \text{count}(w, d')}{\sum_{d' \in C} |d'|}$$

Effect

Even if $\text{count}(w, d) = 0$, we have $P(w|d) > 0$ from collection

Jelinek-Mercer Smoothing

Formula

$$P(w|d) = (1 - \lambda) \cdot P_{\text{ML}}(w|d) + \lambda \cdot P(w|C)$$

Parameter λ (typically 0.1–0.3):

- $\lambda = 0$: Only use document (no smoothing)
- $\lambda = 1$: Only use collection (ignore document)
- $\lambda = 0.1$: Mostly trust document, small collection backup

Properties:

- ✓ Simple and interpretable
- ✓ Same λ for all documents
- ✗ Doesn't adapt to document length

Dirichlet Smoothing

Formula

$$P(w|d) = \frac{\text{count}(w, d) + \mu \cdot P(w|C)}{|d| + \mu}$$

Parameter μ (typically 1000–2500):

- Smoothing strength (number of "pseudo-counts" from collection)
- Higher μ: More smoothing. Lower μ: Less smoothing.

Key Difference: Length-Adaptive

- **Short documents:** μ dominates denominator $\rightarrow$ rely more on collection
- **Long documents:** Actual counts dominate $\rightarrow$ rely on document
- Automatically adapts smoothing to document length!

Dirichlet Smoothing: Length Adaptation Example

Term "algorithm" with $P(w|C) = 0.001$**,** $\mu = 2000$

Document	**count(w,d)**	$\|d\|$	$P(w\|d)$
Short doc	2	100	$\frac{2+2}{2100} = 0.0019$
Long doc	2	2000	$\frac{2+2}{4000} = 0.001$

Observation

- Same term count (2), but short doc $\rightarrow$ higher probability
- Long doc $\rightarrow$ more smoothing, closer to collection probability
- This is **automatic** based on $|d|$

Query Likelihood Scoring: Step by Step

The Recipe

Given query q and document d, score $= \log P(q|d)$:

1. **Estimate** via MLE: $P_{\text{ML}}(w|d) = \dfrac{\text{count}(w, d)}{|d|}$
2. **Smooth** (JM): $\hat{P}(w|d) = (1 - \lambda)\, P_{\text{ML}}(w|d) + \lambda\, P(w|C)$
3. **Score**: $\text{score}(q, d) = \sum_{w \in q} \log \hat{P}(w|d)$
4. **Rank** by score — higher score = more relevant.

Section 4

Query Expansion and Pseudo Relevance Feedback

The Vocabulary Mismatch Problem

Core Issue

Queries and relevant documents use different vocabulary:

Medical Domain

- Query: "heart attack"
- Document: "myocardial infarction"

Technical Domain

- Query: "ML"
- Document: "machine learning"

General Domain

- Query: "elderly care"
- Document: "geriatric nursing"

Result

- BM25 score = 0
- Relevant doc not retrieved

Sparse Retrieval Limitation

Exact term matching only — no semantic understanding

Relevance Feedback Overview

Explicit Relevance Feedback

User marks documents as relevant/non-relevant

- ✗ Impractical: users won't give feedback
- ✗ Chicken-and-egg: need good initial results

Pseudo Relevance Feedback (PRF)

Assumption: Top-k retrieved documents are relevant

- ✓ No user interaction needed
- ✓ Automatic query expansion
- ✗ Risk: If assumption fails $\rightarrow$ query drift

Process:

1. Retrieve initial results with original query
2. Extract terms from top-k documents
3. Expand query with these terms
4. Re-retrieve with expanded query

RM3: Relevance Model 3

Provides a probabilistic way to perform query expansion by building a **model of relevance** (R) from the initial top-k results.

Phase 1: Estimating the Relevance Model $P(w|R)$ We want to find terms w that are likely to appear in relevant documents:

$$P(w|R) \approx \sum_{d \in \text{top-}k} \underbrace{P(w|d)}_{\text{term presence}} \cdot \underbrace{P(d|q)}_{\text{doc relevance weight}}$$

- $P(w|d)$ is the smoothed language model of document d.
- $P(d|q)$ is the score of the document from the first retrieval stage.
- Terms common across many high-scoring documents get the highest $P(w|R)$.

RM3: Interpolation and Final Query

Phase 2: Combining with the Original Query To prevent the query from drifting too far, we interpolate the relevance model with the original query's word distribution:

RM3 Linear Interpolation

$$P(w|q_{\text{new}}) = (1-\lambda)P(w|q_{\text{orig}}) + \lambda P(w|R)$$

- $\lambda = 1$: Full query expansion (total trust in top-k).
- $\lambda = 0$: No expansion (original query only).
- **Common default:** $\lambda \approx 0.5$.

The Final Retrieval

The top `fb_terms` from $P(w|q_{\text{new}})$ are used to form a new weighted query for the second stage.

RM3: Key Parameters

fb_docs (Feedback Documents): 3–10

How many top documents to use for expansion

- Too few: May miss relevant terms. Too many: Risk non-relevant docs.
- **Standard:** 10

fb_terms (Feedback Terms): 10–20

How many expansion terms to add

- Too few: Limited benefit. Too many: Noise.
- **Standard:** 10

λ (Interpolation Weight): 0.5–0.8

Original vs. expansion weight

- Lower (0.5): Trust original query more
- Higher (0.8): Trust expansion more

RM3 Example

Original Query: "machine learning"

Top-3 Retrieved Documents (fb_docs=3):

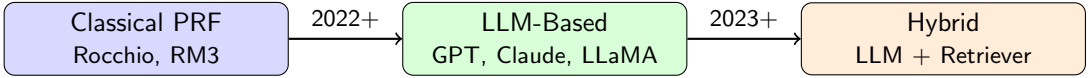
1. "Neural networks and deep learning algorithms..."
2. "Supervised machine learning techniques..."
3. "Machine learning applications in AI..."

Extracted Terms (fb_terms=5):

- neural (P=0.15), deep (P=0.12), supervised (P=0.10), algorithms (P=0.09), AI (P=0.08)

Expanded Query ($\lambda = 0.6$):
"machine learning neural deep supervised algorithms AI"

Beyond PRF: Modern Query Expansion Strategies



Key Insight

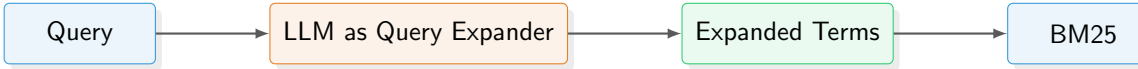
LLMs have **parametric knowledge** — they can generate expansion terms without retrieving any documents first. No risk of PRF's "bad top-k" problem!

Strategy 1: LLM Synonym & Paraphrase Generation — Intuition

Query: "heart attack symptoms" $\longrightarrow$ BM25 score = 0 for a document that says "myocardial infarction"

Core Idea

LLMs were trained on billions of words — they *know* that doctors call a heart attack a "myocardial infarction". Ask the LLM to bridge the vocabulary gap **before** retrieval runs.



Key Advantage over PRF

No documents needed first — LLM's parametric knowledge bridges the gap from the start.

Strategy 1: LLM Synonym & Paraphrase Generation — Method

Prompt:

```
Given the search query:  "heart attack symptoms"
Generate 5 alternative phrasings and medical synonyms that a relevant document might use.
Return only the terms, one per line.
```

LLM Output:

- myocardial infarction signs
- cardiac arrest warning indicators
- chest pain emergency symptoms
- coronary event presentation
- acute heart failure signs

Expanded BM25 query: "heart attack symptoms myocardial infarction cardiac coronary chest pain"

Advantage

Bridges the exact vocabulary gap that BM25 cannot handle on its own

Strategy 2: Hypothetical Document Generation (HyDE)

The Insight

BM25 needs *term overlap* between query and document. A 3-word query has very little overlap. A hypothetical **relevant document** would have much more overlap with real documents.

Core Idea (Gao et al., 2022)

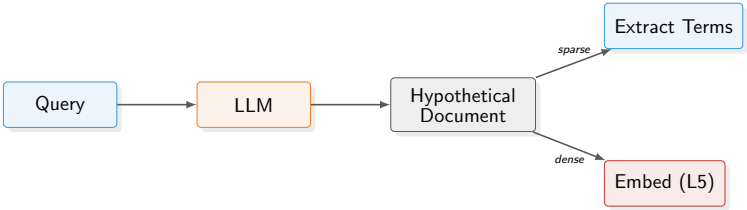
"What if we generated the answer first, then searched for it?"

`https://arxiv.org/pdf/2212.10496`

Strategy 2: Hypothetical Document Generation (HyDE)

Core Idea (Gao et al., 2023)

"What if we generated the answer first, then searched for it?"



Strategy 2: Hypothetical Document Generation (HyDE)

Prompt:

```
Write a short paragraph that would be a perfect answer to the question: "What causes ocean
acidification?"
```

LLM Output:

> *"Ocean acidification is primarily caused by the absorption of atmospheric CO_2 by seawater. When CO_2 dissolves, it forms carbonic acid, which lowers the pH of the ocean. This process threatens marine organisms with calcium carbonate shells, such as corals and mollusks..."*

Sparse path: Extract key terms → "carbonic acid", "calcium carbonate", "pH", "seawater", "corals" → BM25 query

Dense path: Embed the hypothetical doc → use as ANN query vector (Lecture 5)

Strategy 2: Hypothetical Document Generation (HyDE)

Prompt:

```
Write a short paragraph that would be a perfect answer to the question: "What causes ocean
acidification?"
```

LLM Output:

"Ocean acidification is primarily caused by the absorption of atmospheric CO_2 by seawater. When CO_2 dissolves, it forms carbonic acid, which lowers the pH of the ocean. This process threatens marine organisms with calcium carbonate shells, such as corals and mollusks..."

✓ **When it works well:**

- Factual / informational queries
- Domain-specific terminology

✗ **Risk:**

- LLM hallucinations pollute terms
- Adds LLM latency per query

Strategy 3: Chain-of-Thought Query Decomposition — Intuition

The Problem

"How did the 2008 financial crisis affect European housing markets?"
No single document covers all aspects. BM25 on the full query retrieves almost nothing.

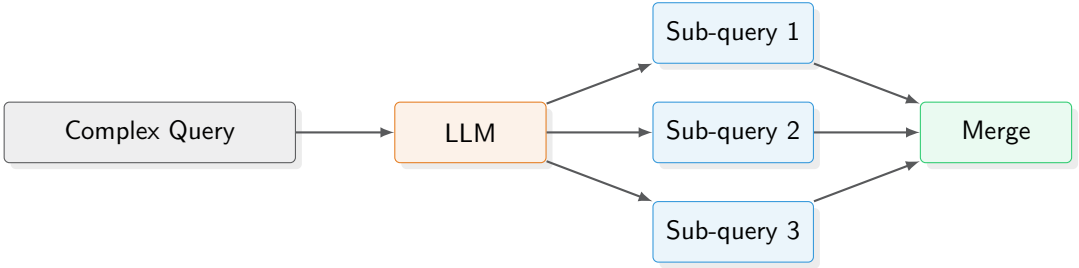
Core Idea

Decompose complex queries into **atomic sub-queries**, retrieve evidence for each independently, then merge the results. More recall, less vocabulary mismatch.

Strategy 3: Chain-of-Thought Query Decomposition — Intuition

Core Idea

Decompose complex queries into **atomic sub-queries**, retrieve evidence for each independently, then merge the results. More recall, less vocabulary mismatch.



Strategy 3: Chain-of-Thought Query Decomposition — Method

Prompt:

```
I need to answer:  "How did the 2008 financial crisis affect European housing markets?"
Break this into 3-4 simpler search queries that together would cover the full answer.
```

LLM Output:

1. "2008 financial crisis causes subprime mortgage"
2. "European housing prices 2008 2009 decline"
3. "EU bank bailouts housing market impact"
4. "European mortgage lending regulations post-crisis"

Pipeline

$\texttt{BM25}_1 \parallel \texttt{BM25}_2 \parallel \texttt{BM25}_3 \parallel \texttt{BM25}_4 \rightarrow$ merge (union / RRF) $\rightarrow$ better recall than single complex query

Best for

Multi-hop reasoning queries, complex informational needs. **Not** for simple keyword lookups.

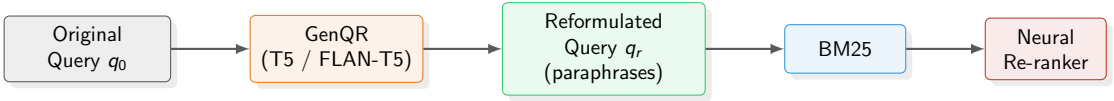
Strategy 4: GenQR (Generative Query Reformulation) — Intuition

The Problem with "term lists" expansions

Synonyms/HyDE/Query2Doc can help, but often produce *bag-of-terms* or long pseudo-text.
Sometimes you just want a **better query** that expresses the same intent.

Core Idea (Wang et al., 2022)

Use an LLM to **rewrite the query itself** into 3–5 high-quality paraphrases, then retrieve using the reformulated query (often appended/interpolated with the original).



Comparison: Classical vs. LLM Query Expansion

Aspect	Classical (RM3)	LLM-Based
Knowledge source	Top-k retrieved docs	Parametric (training data)
Failure mode	Query drift (bad top-k)	Hallucination
Latency	Fast (stats only)	Slower (LLM inference)
Cost	Free	API cost / GPU
Domain knowledge	Needs in-domain corpus	Pre-trained, zero-shot
Cold-start	Fails (empty index)	Works (no index needed)
Interpretability	Terms from real docs	Generated terms

Best Practice (2024+)

Combine both: Use LLM expansion to improve the initial retrieval, then apply RM3 on the (now better) top-k results. The two methods are complementary.

When Query Expansion Helps vs. Hurts

✓ Helps When:

- Initial results mostly relevant
- Vocabulary mismatch present
- Broad informational queries
- Query: "climate change"
- Expansion adds: "warming", "CO2", "emissions"

✗ Hurts When:

- Initial results off-topic
- Ambiguous queries
- Very specific queries
- Query: "jaguar"
- Expansion adds: "car", "speed" (wrong sense!)

Query Drift

Query Drift Example

Query: "python programming"
Top result: Article about Python snake species
RM3 expansion: "reptile", "snake", "habitat", "venom"
LLM expansion: "programming language", "syntax", "libraries" ✓ Correct!

Section 6

Connections to Future Lectures

What We Covered Today

Sparse, Term-Based Ranking

- **BM25:** Probabilistic relevance, saturating TF, length normalization
- **Language Models:** Generative probabilities, smoothing essential
- **Query Expansion:** RM3 for vocabulary mismatch + LLM-based alternatives

Fundamental Limitations

- Still bag-of-words (order doesn't matter)
- Exact term matching only (even with expansion)
- No semantic understanding
- "car" and "automobile" completely different
- "bank" (river) vs. "bank" (financial) treated identically

Appendix: Detailed BM25 Worked Example

Query: "quantum algorithm" $N = 100{,}000$, avgdl $= 500$

Candidate Doc A:

- $|d| = 300$
- tf(quantum) $= 5$
- tf(algorithm) $= 3$

Candidate Doc B:

- $|d| = 800$
- tf(quantum) $= 8$
- tf(algorithm) $= 1$

Stats: df(quantum) $= 200$, df(algorithm) $= 5{,}000$

1. IDF Calculation:

$$\text{IDF(q)} = \log \frac{10^5 - 200 + 0.5}{200 + 0.5} \approx \mathbf{6.21}$$

$$\text{IDF(a)} = \log \frac{10^5 - 5000 + 0.5}{5000 + 0.5} \approx \mathbf{2.94}$$

Appendix: BM25 Worked Example (cont.)

2. Final Scores ($k_1 = 1.2, b = 0.75$):

Step	**Doc A ($\|d\| = 300$)**	**Doc B ($\|d\| = 800$)**
Length Norm	$1 - 0.75 + 0.75(\frac{300}{500}) = \mathbf{0.7}$	$1 - 0.75 + 0.75(\frac{800}{500}) = \mathbf{1.45}$
TF(q) term	$\frac{5 \times 2.2}{5 + 1.2 \times 0.7} = \mathbf{1.88}$	$\frac{8 \times 2.2}{8 + 1.2 \times 1.45} = \mathbf{1.81}$
TF(a) term	$\frac{3 \times 2.2}{3 + 1.2 \times 0.7} = \mathbf{1.72}$	$\frac{1 \times 2.2}{1 + 1.2 \times 1.45} = \mathbf{0.80}$
Total Score	$6.21(1.88) + 2.94(1.72) = \mathbf{16.73}$	$6.21(1.81) + 2.94(0.80) = \mathbf{13.59}$

Insight

Doc A wins because it is concise and covers both query terms effectively. Doc B is penalized for its length and lack of the second term (algorithm).